

SENAI

Curso Técnico em Desenvolvimento de Sistemas

PLANO DE TESTES

Documento de exemplo para a disciplina de Testes de Software

Projeto de referência: VetCare API — Sistema completo de gestão de clínica veterinária

Baseado na norma ISO/IEC/IEEE 29119-3

Este documento é um EXEMPLO. Use-o como modelo, substituindo cada seção pelas informações do projeto da sua equipe.

1. Identificação

Projeto: VetCare API — Sistema de gestão de clínica veterinária (agendamento, prontuário e cobrança)

Versão do sistema em teste: 2.3.0

Versão do plano de testes: 1.0 — 02/09/2026

Responsável pelo plano: Nome do(a) aluno(a) / equipe

Equipe de desenvolvimento: Squad Back-end (API) + integração com gateway de pagamento externo

Aprovador: Professor(a) da disciplina

Contexto: a VetCare API já está em produção com um conjunto reduzido de clínicas-piloto. Este plano cobre a rodada de testes da versão 2.3.0, que adiciona o módulo de pagamentos e amplia o prontuário eletrônico.

2. Itens a testar

A VetCare API é dividida em módulos. A tabela abaixo lista os principais endpoints de cada módulo, com a prioridade de teste.

Módulo	Endpoint / Funcionalidade	Descrição	Prioridade
Autenticação	POST /auth/login	Login de tutores, veterinários e recepção	Alta
Autenticação	POST /auth/refresh-token	Renovação de token JWT expirado	Média
Tutores	POST /tutores	Cadastro de novo tutor (dono do pet)	Média
Tutores	GET /tutores/:id	Consulta de dados de um tutor	Baixa
Pets	POST /pets	Cadastro de pet vinculado a um tutor	Média
Pets	GET /pets/:id/historico	Histórico clínico consolidado do pet	Alta
Veterinários	GET /veterinarios/:id/agenda	Horários disponíveis de um veterinário	Média
Consultas	POST /consultas	Agendamento de nova consulta (fluxo crítico)	Alta
Consultas	PATCH /consultas/:id/status	Confirmação, cancelamento e conclusão da consulta	Alta
Prontuário	POST /consultas/:id/prontuario	Registro de diagnóstico, exames e prescrição	Alta

Pagamentos	POST /pagamentos	Cobrança da consulta via gateway externo	Alta
Pagamentos	GET /pagamentos/:id	Consulta do status de um pagamento	Média
Notificações	POST /notificacoes/lembrete	Disparo de lembrete de consulta/vacina (e-mail e SMS)	Baixa

3. Escopo

Dentro do escopo

- Testes de integração de todos os módulos listados (Autenticação, Tutores, Pets, Veterinários, Consultas, Prontuário, Pagamentos e Notificações), via Jest + Supertest.
- Integração com o gateway de pagamento em ambiente sandbox (não é feita cobrança real).
- Simulação (mock) do provedor de e-mail/SMS para validar o disparo de notificações sem enviar mensagens reais.
- Casos de sucesso, casos de erro esperado (400, 404, 409) e casos de falha de autenticação/autorização (401, 403).

Fora do escopo

- Testes de interface dos aplicativos mobile (tutor) e web (clínica/recepção).
- Testes de carga/performance em produção e testes de segurança ofensiva (pentest).
- Módulo de relatórios administrativos e exportação financeira, ainda em desenvolvimento.
- Integração com o gateway de pagamento em ambiente de produção (fica sob responsabilidade da equipe de operações no deploy).

4. Abordagem / Estratégia

Nível de teste: Teste de integração (caixa-preta) de ponta a ponta, complementado por testes unitários das regras de negócio críticas (ex.: cálculo de conflito de horário)

Técnicas utilizadas: Particionamento de equivalência e análise de valor-limite (datas, horários e valores monetários); teste de tabela de decisão para as regras de autorização por perfil (tutor, veterinário, recepção)

Ferramentas: Jest (test runner), Supertest (requisições HTTP), banco de dados de teste isolado, mock do gateway de pagamento (modo sandbox) e mock do provedor de e-mail/SMS

Tipo de execução: Automatizada, integrada à esteira de CI — a suíte roda a cada Pull Request e novamente antes do deploy em homologação

5. Critérios de entrada

- Todos os endpoints listados na seção 2 implementados e disponíveis em ambiente de homologação.
- Ambiente sandbox do gateway de pagamento configurado com chaves de teste válidas.
- Banco de dados de teste populado com massa mínima: tutores, pets, veterinários e agendas cadastrados.
- Documentação da API (contrato de rotas e modelos de dados) atualizada e disponível para a equipe de testes.

6. Critérios de saída / aprovação

- 100% dos casos de teste planejados foram executados.
- Todos os casos de teste dos módulos de Consultas, Prontuário e Pagamentos (prioridade Alta) foram aprovados.

- Nenhum bug de severidade Alta em aberto (ex.: conflito de agendamento, cobrança duplicada, exposição de dados de outro tutor).
- Cobertura mínima de 80% nos módulos de Consultas e Pagamentos.

7. Critérios de suspensão

Suspender quando: o ambiente de homologação ficar indisponível, o sandbox do gateway de pagamento parar de responder, ou mais de 50% dos casos de teste forem bloqueados por erro 500 generalizado

Como retomar: restabelecer o serviço afetado (API, banco ou sandbox externo), validar a estabilidade com uma execução de fumaça (smoke test) e retomar a partir do ponto de suspensão

8. Entregáveis

- Relatório de execução de testes (casos aprovados x reprovados), gerado pelo pipeline de CI.
- Relatório de cobertura de código por módulo.
- Casos de teste documentados (arquivos .test.js organizados por módulo).
- Relatório de bugs encontrados, com severidade, passos para reprodução e módulo afetado.

9. Cronograma e recursos

Atividade	Responsável	Ferramenta	Prazo
Testes de Autenticação, Tutores e Pets	Aluno(a) A	Jest + Supertest	Dia 1
Testes de Consultas e Prontuário	Aluno(a) B	Jest + Supertest	Dia 2
Testes de Pagamentos (sandbox) e Notificações	Aluno(a) C	Jest + Supertest + mocks	Dia 3
Execução completa da suíte de regressão	Equipe	Jest (npm test) via CI	Dia 4
Consolidação do relatório final	Aluno(a) responsável pelo plano	Documento/planilha	Dia 5

10. Riscos

Cada risco é avaliado por probabilidade x impacto. Nesta seção, o Nível é destacado por cor para facilitar a leitura rápida da equipe (verde = baixo, amarelo = médio, vermelho = alto).

Risco	Probabilidade	Impacto	Nível	Mitigação
Permitir dois agendamentos no mesmo horário para o mesmo veterinário	50%	Alto	Alta	Validação de conflito de horário no banco (constraint) + teste automatizado específico do cenário
Gateway de pagamento (sandbox) instável ou fora do ar durante os testes	30%	Alto	Alta	Criar mock local do gateway para não depender do sandbox externo em toda execução
Falha de autorização expõe dados de pets/consultas de outro tutor (violação de LGPD)	10%	Muito Alto	Alta	Casos de teste dedicados de autorização por perfil, incluindo tentativa de acesso cruzado entre tutores
Reenvio de requisição gera cobrança duplicada no pagamento	30%	Alto	Alta	Implementar chave de idempotência no endpoint de pagamento e testar reenvio da mesma requisição

Serviço de notificação (e-mail/SMS) indisponível e lembretes não são enviados	50%	Moderado	Média	Usar fila de reenvio com tentativa posterior e mock do provedor nos testes automatizados
Apenas uma pessoa da equipe conhece o fluxo de autenticação JWT	30%	Moderado	Média	Fazer pareamento (pair testing) para disseminar o conhecimento entre a equipe
Massa de dados de teste (tutores/pets/consultas) fica corrompida entre execuções	30%	Baixo	Baixa	Resetar (seed) o banco de dados de teste automaticamente antes de cada execução

Lembrete: este plano é um documento vivo — atualize-o sempre que o escopo, o risco ou o prazo do seu projeto mudarem.

Como adaptar este modelo para o seu projeto

- Seção 1 (Identificação): troque pelo nome do seu projeto, sua equipe e a versão real.
- Seção 2 (Itens a testar): liste os módulos e endpoints reais do seu sistema, agrupando por área funcional como no exemplo.
- Seção 3 (Escopo): decida com a equipe o que entra e o que fica de fora — inclusive integrações externas (pagamento, notificação, etc.).
- Seção 4 (Abordagem): descreva as ferramentas, técnicas e mocks que sua equipe realmente vai usar.
- Seções 5 a 8 (Critérios e entregáveis): adapte às regras da sua disciplina/projeto e ao seu pipeline de CI, se houver.
- Seção 9 (Cronograma): distribua as tarefas entre os integrantes da equipe com prazos realistas.
- Seção 10 (Riscos): pense nos riscos de produto (bugs, falhas de segurança, integrações externas) e nos riscos de projeto (prazo, dependência de uma única pessoa, ambiente instável). Use cor apenas no nível, como no exemplo, para dar destaque visual sem poluir o documento.